Глубокое обучение с подкреплением

Модуль 6: Advantage Actor-Critic (A2C)

Курс Hugging Face Deep RL

Полная учебная версия на основе MDX

Источник: курс Deep RL от Hugging Face (оригинальные материалы на английском языке).

Содержание

1 Введение **2**

2 Проблема дисперсии в REINFORCE **3**

3 Advantage Actor-Critic (A2C) **4**
- 3.1 Уменьшение дисперсии с помощью Actor-Critic . . . 4
- 3.2 Процесс обучения Actor-Critic . . . 5
- 3.3 Добавление функции преимущества в Actor-Critic (A2C) . . . 8
- 3.4 Практика A2C в робототехнической симуляции Panda-Gym . . . 9
- 3.5 Модуль 6: A2C в робототехнической симуляции Panda-Gym . . . 9
- 3.6 Обучим наших первых роботов . . . 10

4 Тест **17**
- 4.1 Решения и пояснения к тесту . . . 18

5 Заключение **20**

6 Дополнительная литература **20**
- 6.1 Компромисс между смещением и дисперсией в RL . . . 20
- 6.2 Функции преимущества . . . 20
- 6.3 Actor-Critic . . . 20

1 Введение



Рис. 1: Введение в методы Actor-Critic: обложка курса [оригинал]

В модуле 4 мы познакомились с первым алгоритмом, непосредственно использующим политику, — **REINFORCE**.

В методах на основе политики **мы стремимся оптимизировать саму политику, не используя функцию ценности**. Точнее, REINFORCE относится к подклассу *методов на основе политики*, называемому *методами градиента политики*. В них политика оптимизируется напрямую: **веса оптимальной политики оцениваются и обновляются методом градиентного подъёма**.

Мы увидели, что REINFORCE работает неплохо. Однако для оценки возврата мы применяем метод Монте-Карло (то есть используем целый эпизод), поэтому **оценка градиента политики обладает значительной дисперсией**.

Напомним, что оценка градиента политики задаёт **направление наиболее быстрого увеличения возврата**. Иными словами, она показывает, как менять веса политики, чтобы повысить вероятность действий, приводящих к большому возврату. Дисперсия оценки Монте-Карло, которую мы подробнее рассмотрим в этом модуле, **замедляет обучение, поскольку для её уменьшения требуется много выборок**.

Поэтому сегодня мы изучим **методы Actor-Critic («Актор–Критик»)** — гибридную архитектуру, объединяющую методы на основе ценности и методы на основе политики. Она позволяет стабилизировать обучение за счёт уменьшения дисперсии с помощью двух компонентов:

- *Актора*, определяющего, **как ведёт себя агент** (метод на основе политики);
- *Критика*, оценивающего, **насколько хорошо выбранное действие** (метод на основе ценности).

Мы изучим один из таких гибридных методов — Advantage Actor-Critic (A2C), — **и обучим агента в робототехнической среде с помощью Stable-Baselines3**. Для этого научим:

- Роботизированную руку перемещаться в заданное положение.

Звучит интересно? Тогда начнём!

2 Проблема дисперсии в REINFORCE

В REINFORCE мы хотим **повышать вероятность действий, совершённых в траектории, пропорционально полученному возврату**.

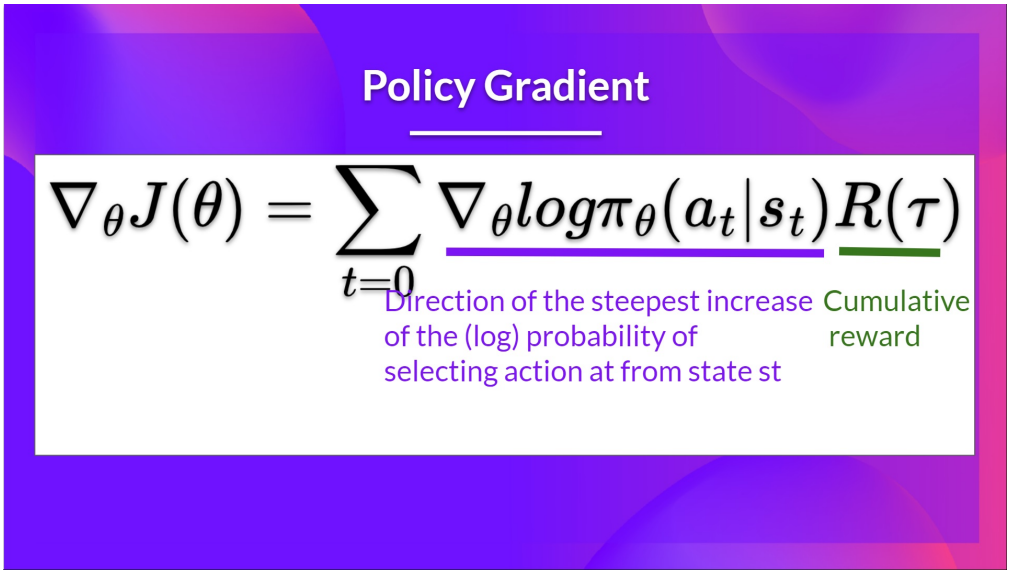


Рис. 2: Обновление градиента политики в REINFORCE [оригинал]

- Если **возврат велик**, мы **увеличиваем** вероятности соответствующих пар (состояние, действие).
- И наоборот: если **возврат мал**, мы **уменьшаем** вероятности соответствующих пар (состояние, действие).

Возврат $R(\tau)$ вычисляется *методом Монте-Карло*. Мы собираем траекторию и рассчитываем дисконтированный возврат, **после чего используем его, чтобы увеличить или уменьшить вероятность каждого действия, выполненного в этой траектории**. Если возврат хороший, все действия траектории «подкрепляются»: вероятность их повторения возрастает.

$R(\tau) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ...$

Преимущество метода заключается в том, что **оценка несмещённая: вместо приближённой оценки возврата** мы используем только фактически полученный возврат.

Из-за стохастичности среды (случайных событий во время эпизода) и самой политики **разные траектории могут давать разные возвраты, что приводит к высокой дисперсии**. Поэтому даже из одного начального состояния мы можем получить сильно различающиеся возвраты. Следовательно, **возврат из одного и того же состояния может значительно изменяться от эпизода к эпизоду**.

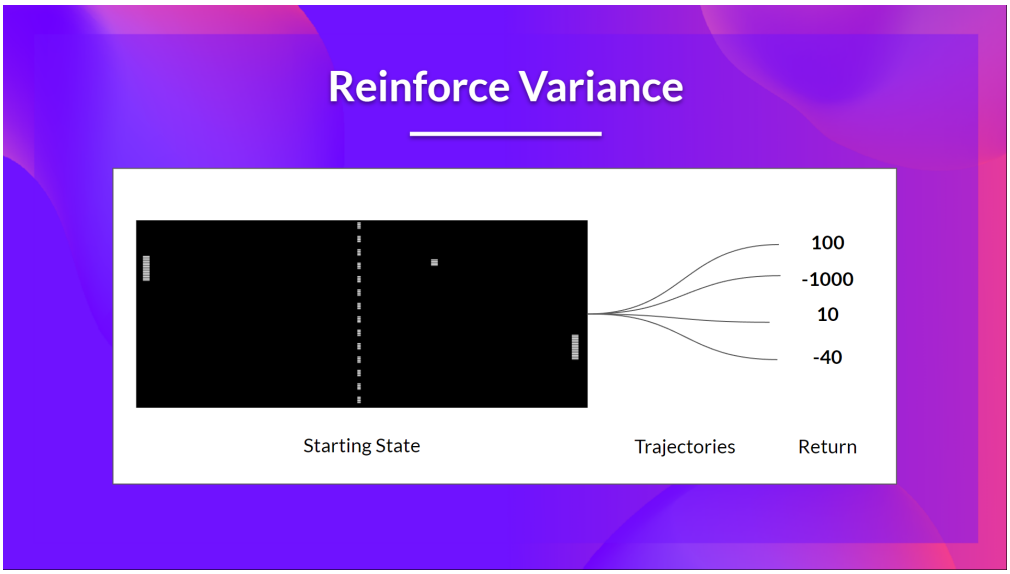


Рис. 3: Дисперсия возвратов Монте-Карло для разных траекторий [оригинал]

Один из способов борьбы с дисперсией — **использовать большое количество траекторий в надежде, что случайные отклонения отдельных траекторий усреднятся и дадут оценку «истинного» возврата**.

Однако значительное увеличение размера пакета **снижает эффективность использования выборок**. Поэтому нужны дополнительные способы уменьшения дисперсии.

Если хочешь подробнее разобраться в компромиссе между смещением и дисперсией в глубоком обучении с подкреплением, прочитай следующие материалы:

- Как понимать компромисс между смещением и дисперсией в глубоком RL
- Компромисс между смещением и дисперсией в обучении с подкреплением
- Высокая дисперсия градиентов политики ***

3 Advantage Actor-Critic (A2C)

3.1 Уменьшение дисперсии с помощью Actor-Critic

Решение, позволяющее уменьшить дисперсию REINFORCE и обучать агента быстрее и устойчивее, состоит в объединении методов на основе политики и методов на основе ценности: это и есть *Actor-Critic*.

Чтобы понять Actor-Critic, представь, что играешь в видеоигру вместе с другом, который даёт тебе обратную связь. Ты — Актор, а твой друг — Критик.

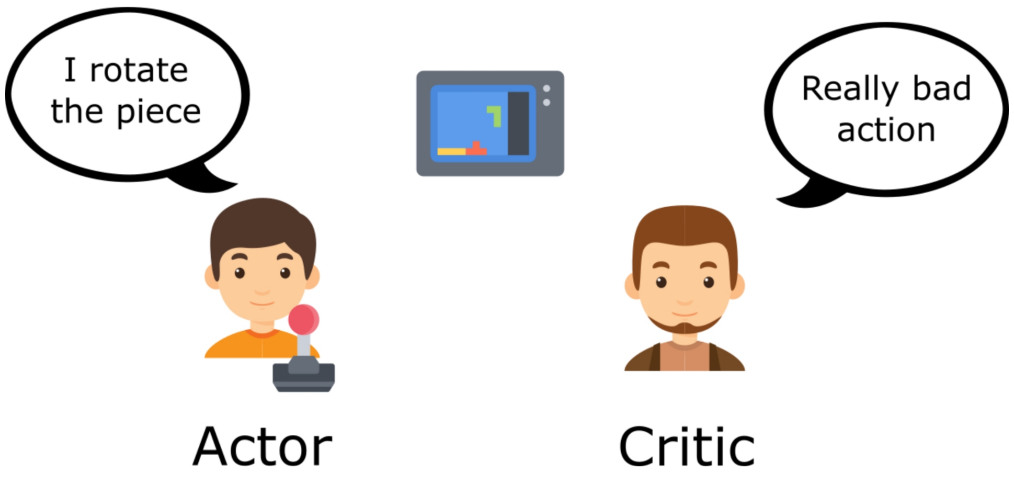


Рис. 4: Актор и Критик: политика и обратная связь от функции ценности [оригинал]

В начале ты ещё не умеешь играть, **поэтому пробуешь случайные действия**. Критик наблюдает за ними и **даёт обратную связь**.

Используя её, **ты обновляешь политику и постепенно начинаешь играть лучше**.

В то же время твой друг (Критик) тоже улучшает способ оценки твоих действий, чтобы в следующий раз давать более качественные советы.

Это основная идея Actor-Critic. Мы обучаем два приближения функций:

- *Политику*, которая **определяет действия агента**: $\pi_\theta(s)$.
- *Функцию ценности*, помогающую обновлять политику и оценивающую, насколько хорошо выбранное действие: $\hat{q}_w(s, a)$.

3.2 Процесс обучения Actor-Critic

Теперь, когда мы разобрали общую идею Actor-Critic, рассмотрим подробнее, как Актор и Критик совместно улучшаются во время обучения.

Как мы уже увидели, в методе Actor-Critic имеются два приближения функций (две нейронные сети):

- *Актор* — **функция политики** с параметрами θ: $\pi_\theta(s)$.
- *Критик* — **функция ценности** с параметрами w: $\hat{q}_w(s, a)$.

Рассмотрим процесс обучения, чтобы понять, как оптимизируются Актор и Критик:

- На каждом шаге t мы получаем из среды текущее состояние S_t и **передаём его на вход Актору и Критику**.
- Политика получает состояние и **выдаёт действие** A_t.

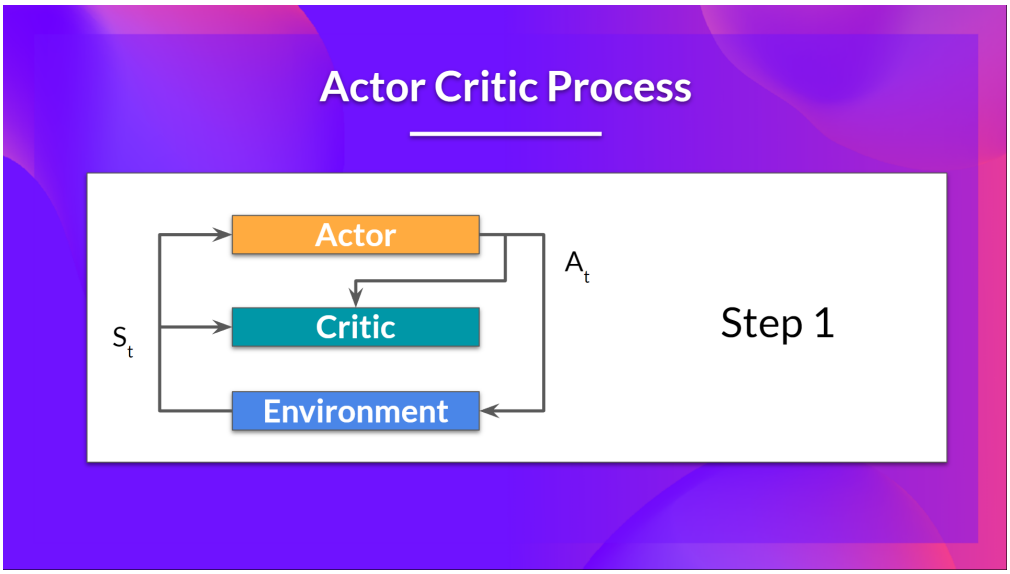


Рис. 5: Процесс обучения Actor-Critic: Актор выбирает действие [оригинал]

- Критик также получает это действие и, используя S_t и A_t, **оценивает ценность выполнения данного действия в данном состоянии — Q-значение**.

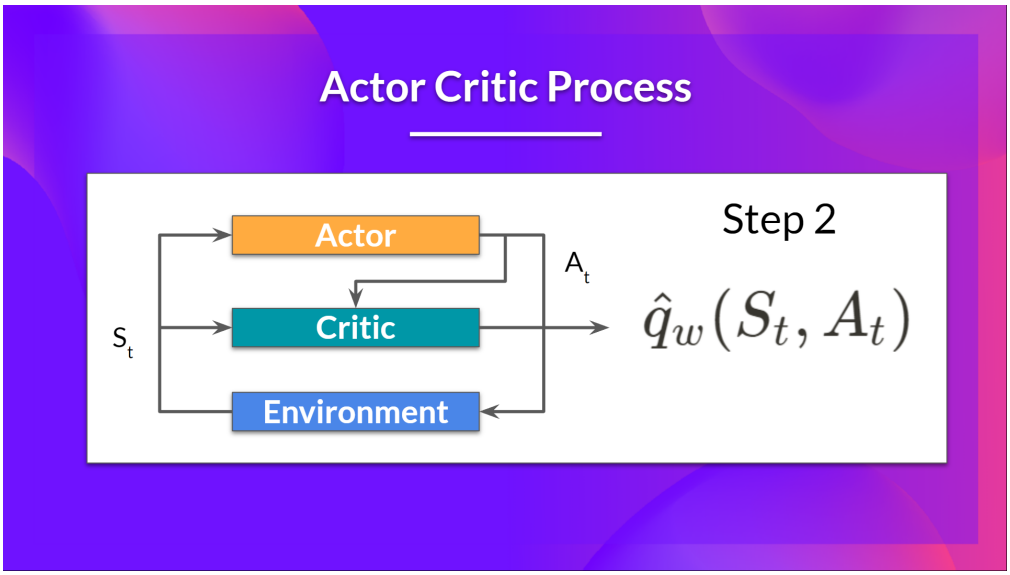


Рис. 6: Процесс Actor-Critic: Критик оценивает пару «состояние–действие» [оригинал]

- После выполнения действия A_t среда возвращает новое состояние S_{t+1} и вознаграждение R_{t+1}.



Рис. 7: Процесс Actor-Critic: среда возвращает вознаграждение и следующее состояние [оригинал]

- Актор обновляет параметры политики, используя Q-значение.

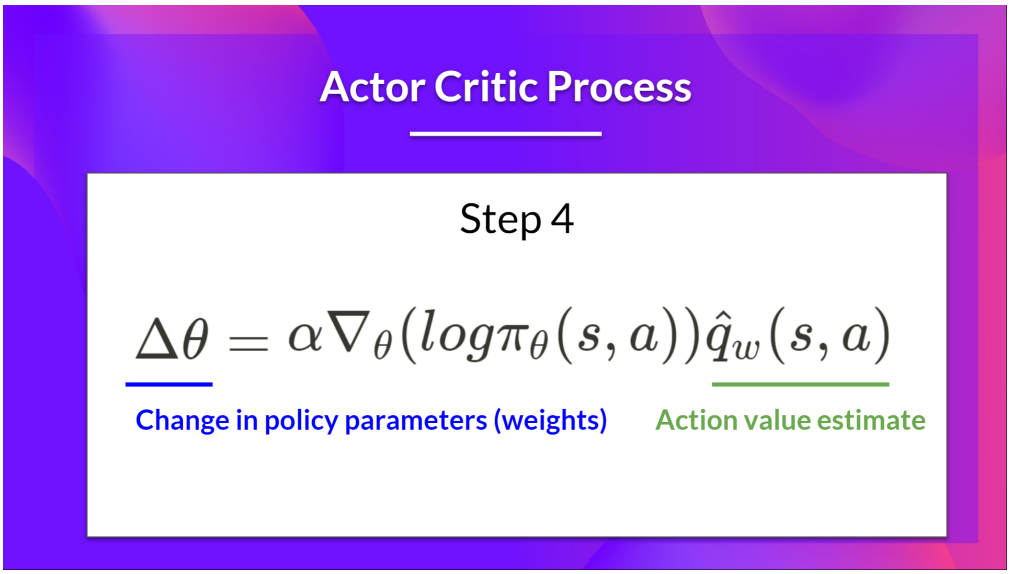


Рис. 8: Процесс Actor-Critic: обновление Актора по Q-значению [оригинал]

- Благодаря обновлённым параметрам Актор выбирает следующее действие A_{t+1}, исходя из нового состояния S_{t+1}.
- Затем Критик обновляет параметры своей функции ценности.

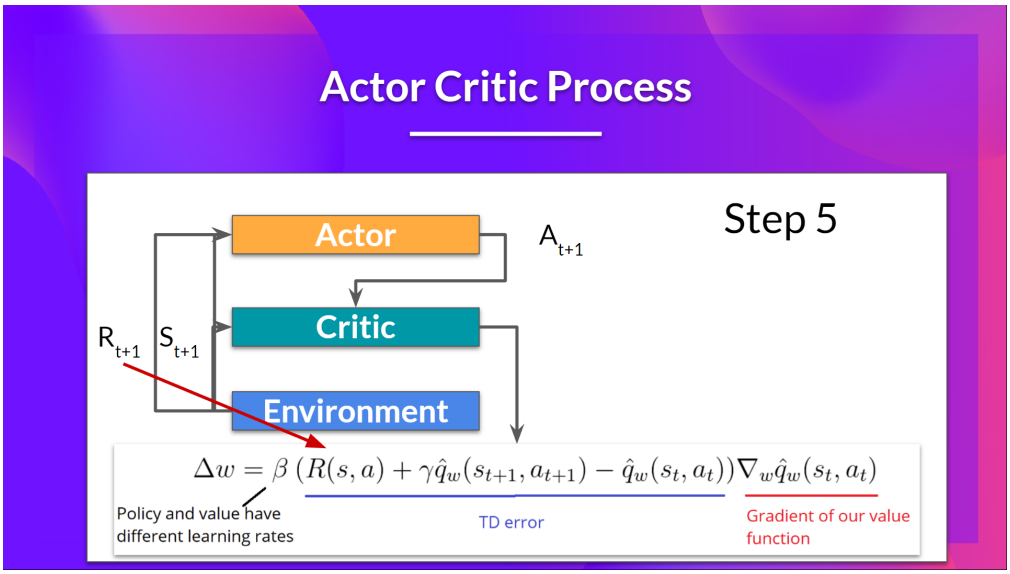


Рис. 9: Процесс Actor-Critic: обновление Критика [оригинал]

3.3 Добавление функции преимущества в Actor-Critic (A2C)

Можно дополнительно стабилизировать обучение, **используя функцию преимущества (Advantage) вместо функции ценности действия для обучения Актора**.

Функция преимущества вычисляет относительную полезность действия по сравнению с другими действиями, доступными в данном состоянии: **насколько выполнение этого действия лучше средней ценности состояния**. Для этого из ценности пары (состояние, действие) вычитается ценность состояния:

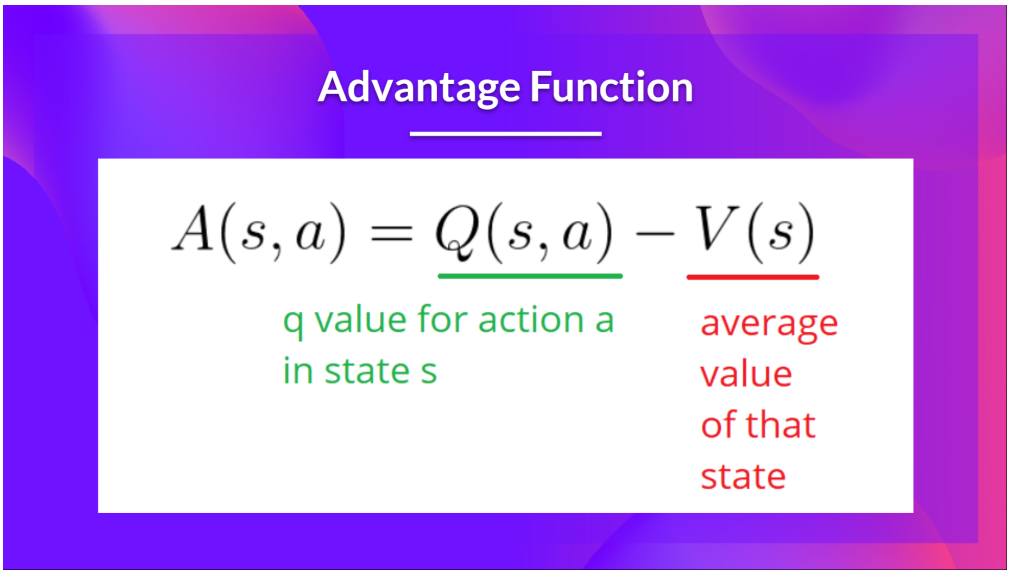


Рис. 10: Определение функции преимущества [оригинал]

Иными словами, она показывает, **какой дополнительный возврат мы получаем от выбранного действия по сравнению со средним возвратом из данного состояния**.

Этот дополнительный возврат — то, что превышает ожидаемую ценность состояния.

- Если $A(s,a) > 0$, градиент **направлен в сторону увеличения вероятности этого действия**.
- Если $A(s,a) < 0$ (действие хуже среднего для данного состояния), **градиент направлен в противоположную сторону**.

Проблема прямого вычисления функции преимущества заключается в необходимости иметь

две функции ценности — $Q(s, a)$ и $V(s)$. К счастью, **TD-ошибка может служить хорошей оценкой функции преимущества**.

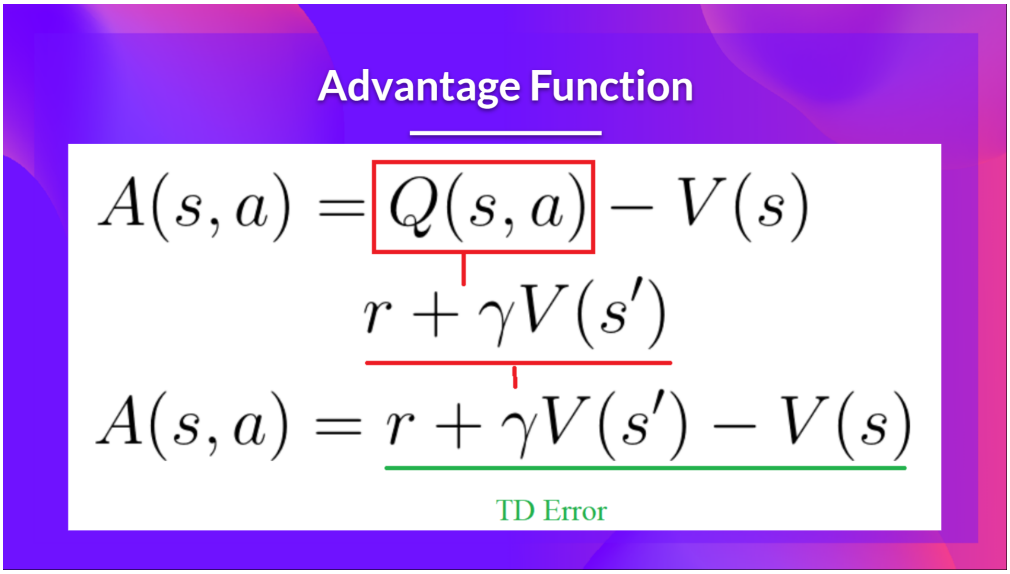


Рис. 11: TD-ошибка как оценка преимущества [оригинал]

3.4 Практика A2C в робототехнической симуляции Panda-Gym

Официальный практический ноутбук модуля 6 (Google Colab)

Помощь сообщества

Теперь, когда ты познакомился с теорией Advantage Actor-Critic (A2C), **ты готов обучить собственного A2C-агента** с помощью Stable-Baselines3 в робототехнической среде. Мы обучим:

- Роботизированную руку перемещаться в заданную точку.

Мы будем использовать:

- panda-gym

Чтобы практическое задание было зачтено для сертификации, необходимо опубликовать обученные модели на Hub и получить следующие результаты:

- Для `PandaReachDense-v3` требуется результат ≥ -3.5.

Чтобы узнать результат, открой таблицу лидеров и найди свою модель. **Результат = mean_reward - std вознаграждения**.

Подробнее о сертификации смотри в разделе https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process.

Чтобы начать практику, нажми кнопку Open In Colab:

Открыть в Google Colab

3.5 Модуль 6: A2C в робототехнической симуляции Panda-Gym

Среды:

- Panda-Gym

Библиотека RL:

- Stable-Baselines3

Мы постоянно стараемся улучшать учебные материалы. Если **ты обнаружишь проблему в ноутбуке**, пожалуйста, создай обращение в репозитории GitHub.

Цели этого ноутбука

После выполнения ноутбука ты сможешь:

- Использовать **библиотеку сред Panda-Gym**.
- **Обучать роботов с помощью A2C**.
- Объяснять, почему **нужна нормализация входных данных**.
- **Публиковать обученного агента и код на Hub**, добавляя видео его работы и оценку качества.

Предварительные требования

Перед началом работы с ноутбуком необходимо:

Изучить методы Actor-Critic в теоретической части модуля 6.

3.6 Обучим наших первых роботов

Настройка GPU

- Чтобы **ускорить обучение агента, используем GPU**. В Colab для этого выбери `Runtime > Change Runtime type`.

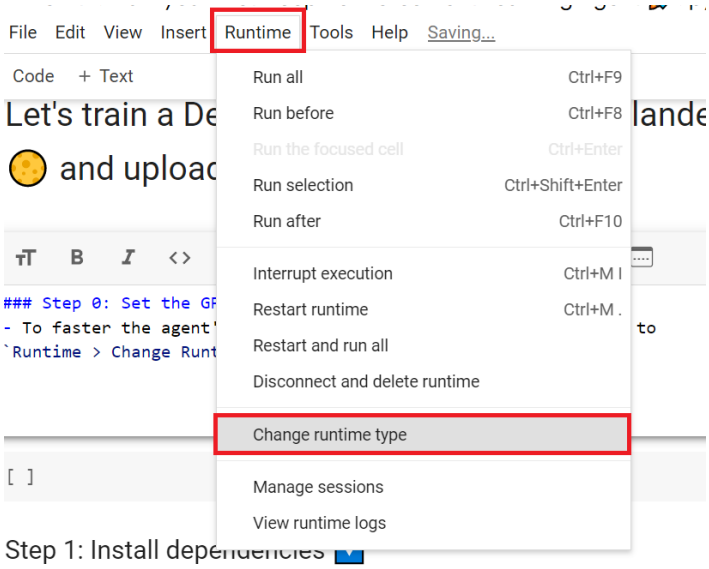


Рис. 12: Google Colab: выбор настроек среды выполнения [оригинал]

- Затем выбери `Hardware Accelerator > GPU`.

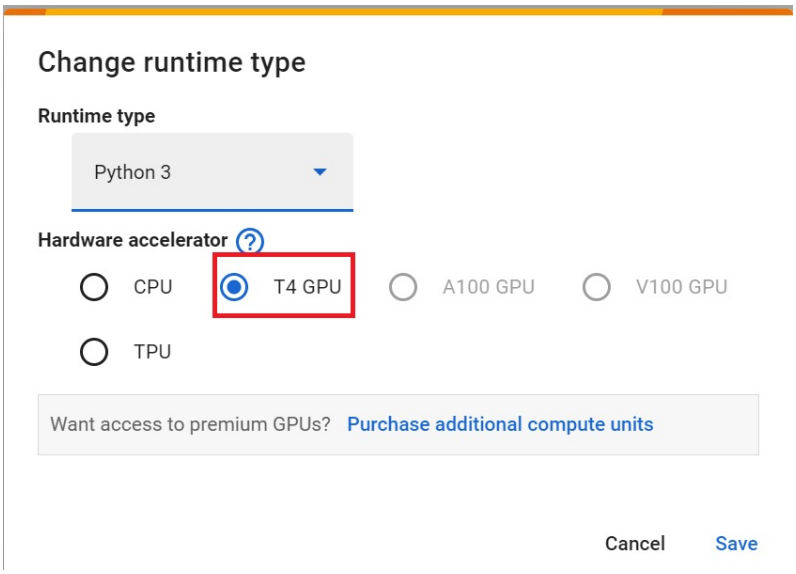


Рис. 13: Google Colab: включение ускорения на GPU [оригинал]

Создание виртуального дисплея

В ходе работы с ноутбуком понадобится создавать видео с демонстрацией агента. Для этого в Colab **нужен виртуальный экран, чтобы отрисовывать среду** и записывать кадры.

Следующая ячейка установит библиотеки, создаст виртуальный экран и запустит его.

```
%%capture
!apt install python-opengl
!apt install ffmpeg
!apt install xvfb
!pip3 install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Установка зависимостей Установим несколько библиотек:

- `gymnasium`.
- `panda-gym`: содержит робототехнические среды с манипуляторами.
- `stable-baselines3`: библиотека алгоритмов глубокого обучения с подкреплением SB3.
- `huggingface_sb3`: дополнительный код для загрузки и публикации моделей Stable-Baselines3 на Hugging Face Hub.
- `huggingface_hub`: библиотека для работы с репозиториями Hugging Face Hub.

```
!pip install stable-baselines3[extra]
!pip install gymnasium
!pip install huggingface_sb3
!pip install huggingface_hub
!pip install panda_gym
```

Импорт пакетов

```
import os

import gymnasium as gym
import panda_gym

from huggingface_sb3 import load_from_hub, package_to_hub

from stable_baselines3 import A2C
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
from stable_baselines3.common.env_util import make_vec_env

from huggingface_hub import notebook_login
```

PandaReachDense-v3

Агент, которого мы обучим, — это роботизированная рука, которой нужно управлять: перемещать манипулятор и его конечный исполнительный орган.

В робототехнике *конечный исполнительный орган (end-effector)* — устройство на конце манипулятора, предназначенное для взаимодействия с окружающей средой.

В `PandaReach` робот должен поместить конечный исполнительный орган в целевую точку (зелёный шар).

Мы используем версию среды с *плотной функцией вознаграждения (dense reward)*: **вознаграждение выдаётся на каждом шаге** (чем ближе агент к выполнению задачи, тем оно больше). Напротив, при *разреженном вознаграждении (sparse reward)* среда **выдаёт награду только после завершения задачи**.

Также будем использовать *управление смещением конечного исполнительного органа*: **действие задаёт его перемещение**, а не движение каждого отдельного сустава (joint control).

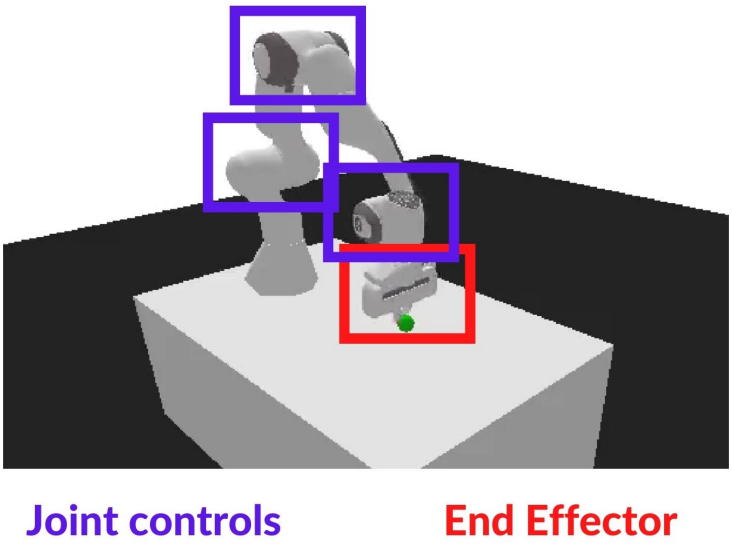


Рис. 14: Робототехническая среда PandaReach [оригинал]

Так **обучение будет проще**.

Создание среды

Среда В `PandaReachDense-v3` робот должен переместить конечный исполнительный орган в целевую точку (зелёный шар).

```
env_id = "PandaReachDense-v3"

# Create the env
env = gym.make(env_id)

# Get the state space and action space
s_size = env.observation_space.shape
a_size = env.action_space
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample()) # Get a random observation
```

Пространство наблюдений **представляет собой словарь из трёх элементов**:

- `achieved_goal`: положение цели по координатам (x, y, z).
- `desired_goal`: расстояние между целевым положением и текущим положением объекта по координатам (x, y, z).
- `observation`: положение (x, y, z) и скорость конечного исполнительного органа (v_x, v_y, v_z).

Поскольку наблюдение представлено словарём, **вместо `MlpPolicy` нам требуется политика `MultiInputPolicy`**.

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample()) # Take a random action
```

Пространство действий — вектор из трёх значений:

- Управление перемещением по координатам x, y, z.

Нормализация наблюдений и вознаграждений В обучении с подкреплением хорошей практикой считается нормализация входных признаков.

Для этого существует обёртка, вычисляющая скользящие оценки среднего значения и стандартного отклонения входных признаков.

Этой же обёрткой нормализуем и вознаграждения, добавив `norm_reward = True`.

Обратись к документации, чтобы заполнить эту ячейку.

```
env = make_vec_env(env_id, n_envs=4)

# Adding this wrapper to normalize the observation and the reward
env = # TODO: Add the wrapper
```

Решение

```
env = make_vec_env(env_id, n_envs=4)

env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)
```

Создание модели A2C Подробнее о реализации A2C в Stable-Baselines3 смотри: https://stable-baselines3.readthedocs.io/en/master/modules/a2c.html#notes.

Для выбора подходящих параметров я обратился к официальным обученным агентам команды Stable-Baselines3.

```
model = # Create the A2C model and try to find the best parameters
```

Решение

```
model = A2C(policy = "MultiInputPolicy",
            env = env,
            verbose=1)
```

Обучение A2C-агента

- Обучим агента в течение 1 000 000 шагов. Не забудь включить GPU в Colab. Это займёт примерно 25–40 минут.

```
model.learn(1_000_000)
```

```
# Save the model and  VecNormalize statistics when saving the agent
model.save("a2c-PandaReachDense-v3")
env.save("vec_normalize.pkl")
```

Оценка агента

- После обучения агента необходимо **проверить качество его работы**.
- Для этого Stable-Baselines3 предоставляет функцию `evaluate_policy`.

```
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaReachDense-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)

# We need to override the render_mode
eval_env.render_mode = "rgb_array"

#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False

# Load the agent
model = A2C.load("a2c-PandaReachDense-v3")

mean_reward, std_reward = evaluate_policy(model, eval_env)

print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")
```

Публикация обученной модели на Hub Убедившись в хороших результатах обучения, мы можем опубликовать обученную модель на Hub одной строкой кода.

Документация библиотеки: https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face--x-stable-baselines3-v20.

С помощью `package_to_hub`, как мы уже обсуждали в предыдущих модулях, **можно оценить агента, записать видео его работы, создать карточку модели и опубликовать всё на Hub**.

Это позволяет:

- **Показать результаты своей работы**.
- **Посмотреть, как агент играет**.
- **Поделиться агентом с сообществом**, чтобы другие пользователи могли его применять.
- **Сравнить качество агента с результатами других учащихся в таблице лидеров**: https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard.

Чтобы поделиться моделью с сообществом, нужно выполнить ещё три шага:

1. Если у тебя ещё нет аккаунта, зарегистрируйся на Hugging Face: https://huggingface.co/join.
2. Войди в аккаунт и сохрани токен аутентификации с сайта Hugging Face.

- Создай новый токен (https://huggingface.co/settings/tokens) **с правом записи (write)**.

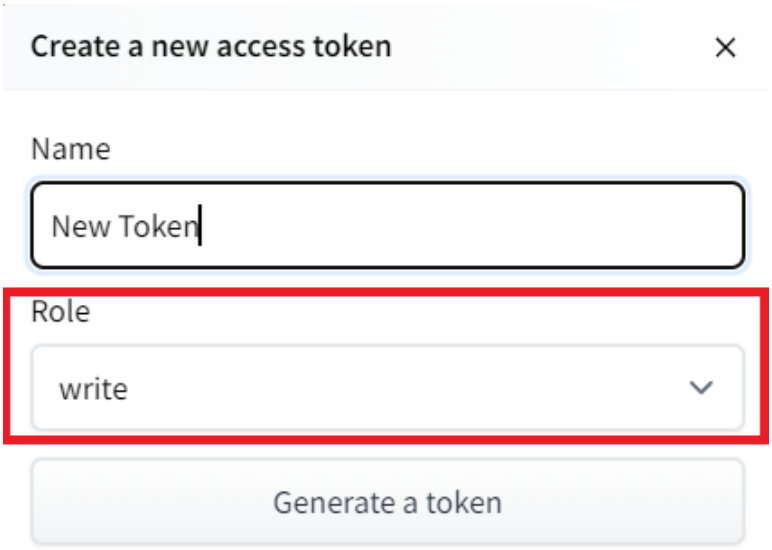


Рис. 15: Создание токена Hugging Face с правом записи [оригинал]

- Скопируй токен.
- Запусти ячейку ниже и вставь токен.

```
notebook_login()
!git config --global credential.helper store
```

Если не используешь Google Colab или Jupyter Notebook, вместо этого выполни команду `huggingface-cli login`.

3. Теперь мы готовы опубликовать обученного агента на Hub с помощью `package_to_hub()`. Для этой среды **выполнение ячейки может занимать примерно 10 минут**.

```
from huggingface_sb3 import package_to_hub

package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
```

```
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # Change the username
    commit_message="Initial commit",
)
```

Дополнительные задания

Лучший способ учиться — **пробовать самостоятельно**! Например, можно попробовать среду `PandaPickAndPlace-v3`.

Если захочешь решать более сложные задачи в panda-gym, изучи примеры с **TQC или SAC** — алгоритмами с более эффективным использованием выборок, подходящими для робототехники. На реальном роботе это особенно важно: в отличие от симуляции, **слишком активные движения манипулятора могут привести к его поломке**.

PandaPickAndPlace-v1 (для этой модели используется версия среды v1): https://huggingface.co/sb3/tqc-PandaPickAndPlace-v1.

Также не забудь изучить документацию panda-gym: https://panda-gym.readthedocs.io/en/latest/usage/train_with_sb3.html.

Ниже перечислены шаги для обучения ещё одного агента (необязательное задание):

1. Создай среду с именем «PandaPickAndPlace-v3».
2. Создай векторизованную среду.
3. Добавь обёртку для нормализации наблюдений и вознаграждений. Смотри документацию.
4. Создай модель A2C (не забудь `verbose=1`, чтобы выводить журнал обучения).
5. Обучи её на 1 млн шагов.
6. Сохрани модель и статистики VecNormalize при сохранении агента.
7. Оцени работу агента.
8. Опубликуй обученную модель на Hub с помощью `package_to_hub`.

Решение (необязательно)

```
# 1 - 2
env_id = "PandaPickAndPlace-v3"
env = make_vec_env(env_id, n_envs=4)

# 3
env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)

# 4
model = A2C(policy = "MultiInputPolicy",
            env = env,
            verbose=1)
# 5
model.learn(1_000_000)
```

```
# 6
model_name = "a2c-PandaPickAndPlace-v3";
model.save(model_name)
env.save("vec_normalize.pkl")

# 7
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Load the saved statistics
eval_env = DummyVecEnv([lambda: gym.make("PandaPickAndPlace-v3")])
eval_env = VecNormalize.load("vec_normalize.pkl", eval_env)
```

```
#  do not update them at test time
eval_env.training = False
# reward normalization is not needed at test time
eval_env.norm_reward = False

# Load the agent
model = A2C.load(model_name)

mean_reward, std_reward = evaluate_policy(model, eval_env)

print(f"Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}")

# 8
package_to_hub(
    model=model,
    model_name=f"a2c-{env_id}",
    model_architecture="A2C",
    env_id=env_id,
    eval_env=eval_env,
    repo_id=f"ThomasSimonini/a2c-{env_id}", # TODO: Change the username
    commit_message="Initial commit",
)
```

До встречи в модуле 7!

Продолжай учиться и развиваться

4 Тест

Лучший способ усвоить материал и избежать иллюзии компетентности — **проверить себя**. Так ты поймёшь, **какие темы нужно повторить**.

Q1: Какая интерпретация компромисса между смещением и дисперсией наиболее точна для обучения с подкреплением?

- **A.** Компромисс между смещением и дисперсией отражает способность модели обобщать знания на ранее размеченные данные, которые мы используем при обучении.
- **B.** Компромисс между смещением и дисперсией отражает, насколько точно сигнал подкрепления соответствует истинному вознаграждению, которое агент должен получать от среды.

Q2: Какие утверждения о смещении и дисперсии моделей в RL верны?

- **A.** Несмещённый сигнал вознаграждения возвращает значения, близкие к реальным или ожидаемым вознаграждениям среды.
- **B.** Смещённый сигнал вознаграждения возвращает значения, близкие к реальным или ожидаемым вознаграждениям среды.
- **C.** Сигнал вознаграждения с высокой дисперсией содержит сильный шум и зависит, например, от случайных (непостоянных) факторов среды.
- **D.** Сигнал вознаграждения с низкой дисперсией содержит сильный шум и зависит, например, от случайных (непостоянных) факторов среды.

Q3: Какие утверждения о методе Монте-Карло верны?

- **A.** Это метод выборочного оценивания: мы рассматриваем не все возможные состояния, а некоторую их выборку.
- **B.** Он очень устойчив к стохастичности (случайным событиям внутри траектории).
- **C.** Чтобы уменьшить влияние случайности в методе Монте-Карло, мы берём `n` траекторий и усредняем их, уменьшая вклад каждой отдельной траектории.

Q4: Как бы ты своими словами описал метод Actor-Critic (A2C)?

Q5: Какие утверждения о методе Actor-Critic верны?

- **A.** Критик не обучает никакую функцию в ходе тренировки.
- **B.** Актор обучает функцию политики, а Критик — функцию ценности.
- **C.** Метод повышает устойчивость к стохастичности и уменьшает высокую дисперсию.

Q6: Что такое `Advantage` в A2C?

Поздравляем с прохождением теста! Если какие-то моменты остались непонятными, удели время повторному чтению главы и закреплению знаний.

4.1 Решения и пояснения к тесту

Решение Q1

Правильный ответ: B.

A. Компромисс между смещением и дисперсией отражает способность модели обобщать знания на ранее размеченные данные, которые мы используем при обучении. Объяснение: это классический компромисс между смещением и дисперсией в машинном обучении. В нашем случае обучения с подкреплением у нас нет заранее размеченных данных — есть только сигнал вознаграждения.

B. Компромисс между смещением и дисперсией отражает, насколько точно сигнал подкрепления соответствует истинному вознаграждению, которое агент должен получать от среды.

Решение Q2

Правильные ответы: A, C.

A. Несмещённый сигнал вознаграждения возвращает значения, близкие к реальным или ожидаемым вознаграждениям среды.

B. Смещённый сигнал вознаграждения возвращает значения, близкие к реальным или ожидаемым вознаграждениям среды. Объяснение: если сигнал вознаграждения смещён, получаемое значение отличается от истинного вознаграждения, которое должна выдавать среда.

C. Сигнал вознаграждения с высокой дисперсией содержит сильный шум и зависит, например, от случайных (непостоянных) факторов среды.

D. Сигнал вознаграждения с низкой дисперсией содержит сильный шум и зависит, например, от случайных (непостоянных) факторов среды. Объяснение: при низкой дисперсии сигнал меньше зависит от шума и даёт близкие значения независимо от случайных факторов среды.

Решение Q3

Правильные ответы: A, C.

A. Это метод выборочного оценивания: мы рассматриваем не все возможные состояния, а некоторую их выборку.

B. Он очень устойчив к стохастичности (случайным событиям внутри траектории). Объяснение: метод Монте-Карло каждый раз случайным образом оценивает выборку траекторий. Однако даже одинаковые траектории могут иметь разный возврат, если содержат случайные события.

C. Чтобы уменьшить влияние случайности в методе Монте-Карло, мы берём `n` траекторий и усредняем их, уменьшая вклад каждой отдельной траектории.

Решение Q4

Идея Actor-Critic состоит в том, что мы обучаем два приближения функций:

1. `policy` — политику, которая определяет действия агента (π);

2. `value` — функцию ценности, помогающую обновлять политику и оценивающую качество выбранного действия (q).

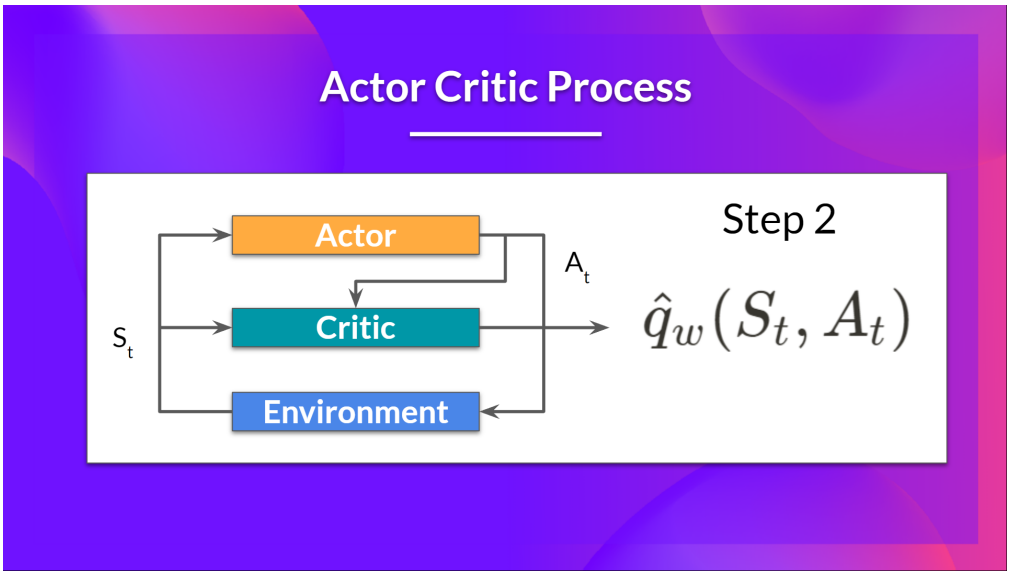


Рис. 16: Процесс Actor-Critic: Критик оценивает пару «состояние–действие» [оригинал]

Решение Q5

Правильные ответы: B, C.

A. Критик не обучает никакую функцию в ходе тренировки. Объяснение: параметры функций и Актора, и Критика обновляются во время обучения.

B. Актор обучает функцию политики, а Критик — функцию ценности.

C. Метод повышает устойчивость к стохастичности и уменьшает высокую дисперсию.

Решение Q6

Вместо прямого использования функции ценности действия Критика можно применять функцию `Advantage` (преимущества). Она оценивает относительное преимущество выбранного действия по сравнению с другими действиями, доступными в данном состоянии, с учётом их среднего результата.

Иными словами, насколько это действие в данном состоянии лучше средней ценности состояния.

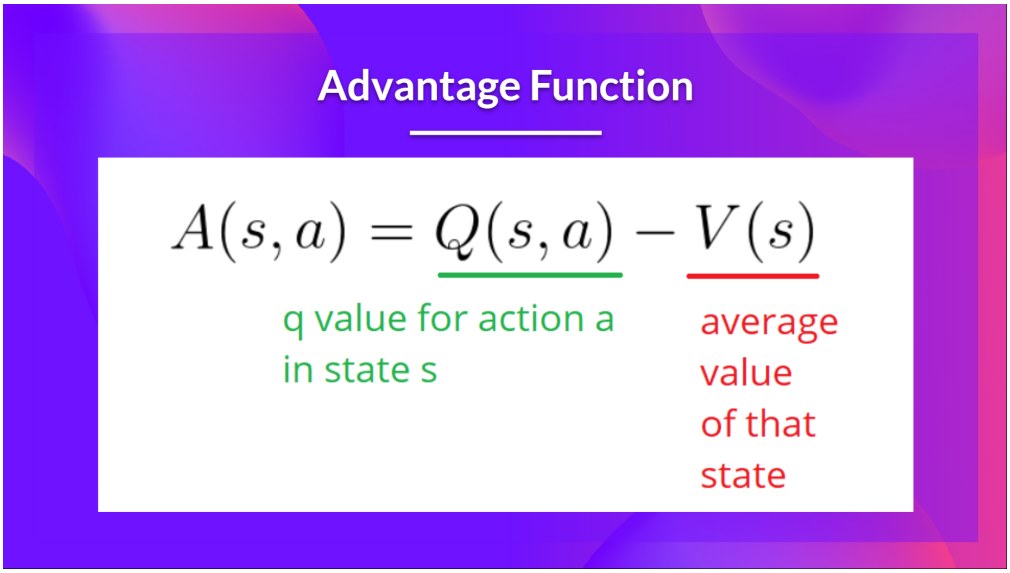


Рис. 17: Определение функции преимущества [оригинал]

5 Заключение

Поздравляем с завершением модуля и практического занятия! Ты только что обучил своих первых виртуальных роботов.

Перед продолжением убедись, что усвоил материал. Также можешь изучить материалы из раздела *«Дополнительная литература»*.

Нам будет очень интересно **узнать твоё мнение о курсе и о том, как его улучшить**. Если есть обратная связь, пожалуйста, заполни эту форму.

До встречи в следующем модуле!

Продолжай учиться и развиваться

6 Дополнительная литература

6.1 Компромисс между смещением и дисперсией в RL

Если хочешь глубже разобраться в компромиссе между смещением и дисперсией в глубоком обучении с подкреплением, посмотри следующие статьи:

- Как понимать компромисс между смещением и дисперсией в глубоком RL
- Компромисс между смещением и дисперсией в обучении с подкреплением

6.2 Функции преимущества

- Функции преимущества: Spinning Up RL

6.3 Actor-Critic

- Основы глубокого RL: градиенты политики и оценка преимущества, лекция Питера Аббила
- Статья по A2C: «Асинхронные методы глубокого обучения с подкреплением»